39. Design Distributed Job Scheduler

Key Terms:

Term	Full Name	What it does
Cron	Time-based scheduler	Unix schedule format (e.g., `0 0 * * *` = daily midnight)
DLQ	Dead Letter Queue	Queue for jobs that failed all retries
Idempotency	Same result on retry	Operation safe to run multiple times
Leader Election	Single active instance	Only one scheduler triggers jobs
Optimistic Locking	Concurrency control	UPDATE WHERE version=N prevents conflicts
Backoff	Retry delay	Increasing wait between retry attempts

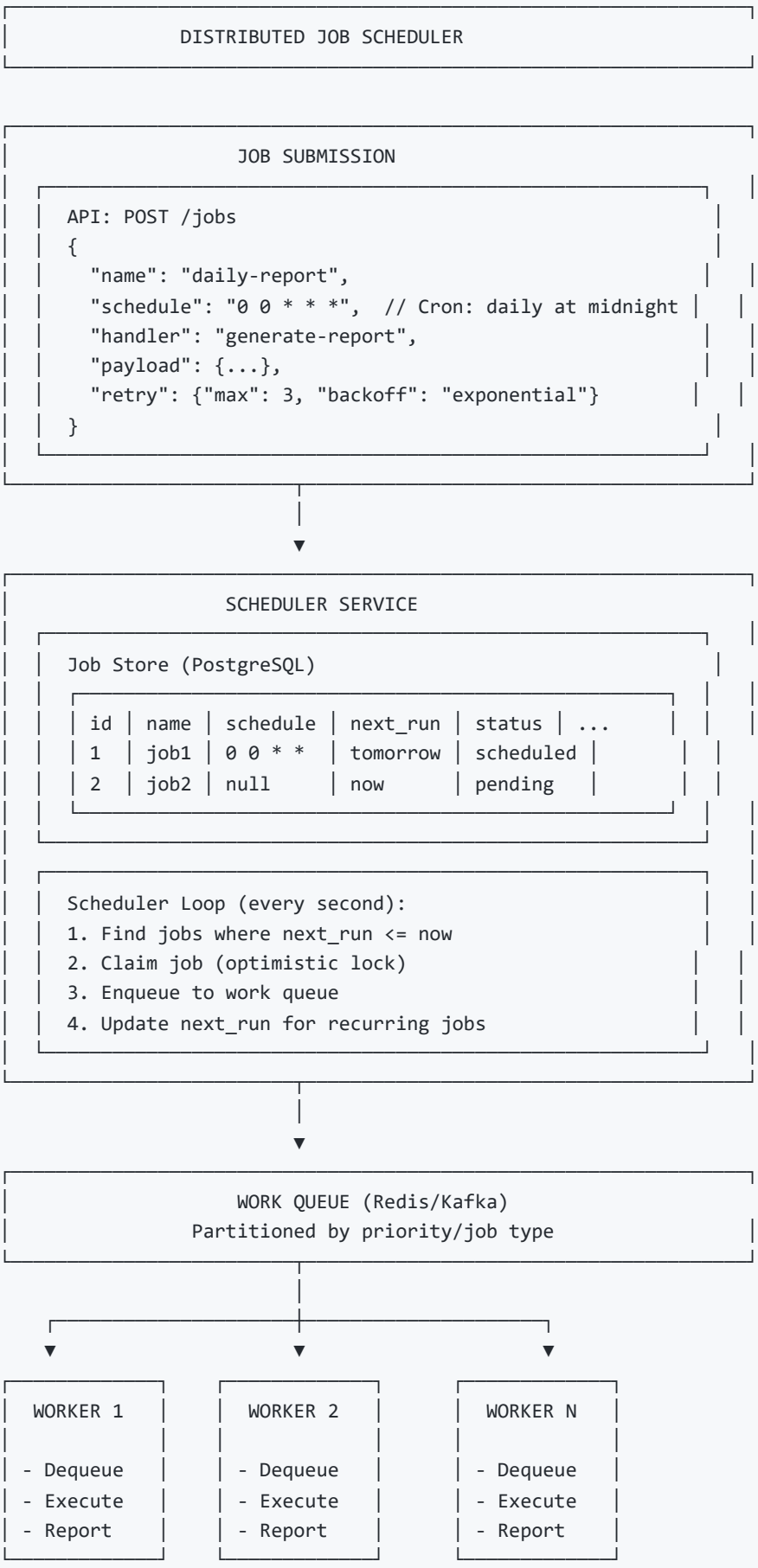
Problem Statement

Design a system that schedules and executes jobs across a cluster of workers with support for cron schedules, one-time jobs, and retries.

Clarifying Questions to Ask

1. **Scale**: How many jobs? Execution frequency?
2. **Job types**: Scheduled, one-time, recurring?
3. **Duration**: Short tasks or long-running jobs?
4. **Reliability**: At-least-once or exactly-once execution?
5. **Priority**: Support different priority levels?
6. **Dependencies**: Jobs depend on other jobs?
7. **Visibility**: Track job status and history?

High-Level Design


DISTRIBUTED JOB SCHEDULER
JOB SUBMISSION
API: POST /jobs
{
"name": "daily-report",
"schedule": "0 0 * * *", // Cron: daily at midnight
"handler": "generate-report",
"payload": {...},
"retry": {"max": 3, "backoff": "exponential"}
}
SCHEDULER SERVICE
Job Store (PostgreSQL)
id | name | schedule | next_run | status | ...
1 | job1 | 0 0 * * | tomorrow | scheduled |
2 | job2 | null | now | pending |
Scheduler Loop (every second):
1. Find jobs where next_run <= now
2. Claim job (optimistic lock)
3. Enqueue to work queue
4. Update next_run for recurring jobs
WORK QUEUE (Redis/Kafka)
Partitioned by priority/job type
WORKER 1
- Dequeue
- Execute
- Report
WORKER 2
- Dequeue
- Execute
- Report
WORKER N
- Dequeue
- Execute
- Report

API Design

Job APIs

Method	Endpoint	Description
`POST`	`/api/v1/jobs`	Create/schedule a job
`GET`	`/api/v1/jobs/{jobId}`	Get job details
`PUT`	`/api/v1/jobs/{jobId}`	Update job configuration
`DELETE`	`/api/v1/jobs/{jobId}`	Delete a job
`POST`	`/api/v1/jobs/{jobId}/trigger`	Manually trigger a job
`POST`	`/api/v1/jobs/{jobId}/pause`	Pause a scheduled job
`POST`	`/api/v1/jobs/{jobId}/resume`	Resume a paused job

Create Job Request

```
POST /api/v1/jobs
{
  "name": "daily-report-generator",
  "type": "recurring",
  "schedule": "0 2 * * *",        // cron expression
  "handler": "report-service",
  "payload": { "report_type": "daily_summary" },
  "retry_policy": {
    "max_retries": 3,
    "backoff": "exponential"
  },
  "timeout_seconds": 300,
  "priority": "high"
}
```

Execution APIs

Method	Endpoint	Description
`GET`	`/api/v1/jobs/{jobId}/executions`	List past executions
`GET`	`/api/v1/executions/{execId}`	Get execution details
`GET`	`/api/v1/executions/{execId}/logs`	Get execution logs

`POST`	`/api/v1/executions/{execId}/cancel`	Cancel running execution

Key Components

Component	Purpose	Technology
API Service	Job CRUD operations	REST API
Job Store	Persist job definitions	PostgreSQL
Scheduler	Trigger due jobs	Singleton or leader election
Work Queue	Buffer jobs for workers	Redis or Kafka
Workers	Execute jobs	Stateless pods
Dead Letter Queue	Failed jobs for review	Separate queue

Exactly-Once Execution

```
CHALLENGE: Multiple schedulers might trigger same job

SOLUTION 1: Leader Election
┌──────────────────────────────────────────────────────────────┐
│  Only one scheduler instance is active (leader)              │
│  Others are standby, take over if leader fails               │
│  Use: ZooKeeper, etcd, or Redis SETNX                        │
└──────────────────────────────────────────────────────────────┘

SOLUTION 2: Optimistic Locking
┌──────────────────────────────────────────────────────────────┐
│  UPDATE jobs                                                 │
│  SET status = 'claimed', claimed_by = 'scheduler-1'          │
│  WHERE id = 123                                              │
│    AND status = 'pending'                                    │
│    AND next_run <= NOW();                                    │
│                                                              │
│  Only one scheduler wins the UPDATE                          │
└──────────────────────────────────────────────────────────────┘

SOLUTION 3: Idempotency Keys
┌──────────────────────────────────────────────────────────────┐
│  Each job execution has unique key: job_id + scheduled_time  │
│  Workers check: "Have I already processed this execution?"   │
│  Dedup store (Redis): SETNX execution:job1:2024-01-15T00:00  │
└──────────────────────────────────────────────────────────────┘
```

Retry and Failure Handling

```
RETRY STRATEGY:
┌─────────────────────────────────────────────────────────────┐
│  {                                                          │
│    "max_retries": 3,                                        │
│    "backoff": "exponential",                                │
│    "initial_delay": "1s",                                   │
│    "max_delay": "1h"                                        │
│  }                                                          │
│                                                             │
│  Attempt 1: Immediate                                       │
│  Attempt 2: 1 second later                                  │
│  Attempt 3: 2 seconds later                                 │
│  Attempt 4: 4 seconds later (if more retries)               │
│                                                             │
│  After max retries: Move to Dead Letter Queue               │
└─────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Scheduler HA**: Leader election or sharded by job ranges
- **Worker scaling**: Auto-scale based on queue depth
- **Partitioning**: Separate queues for different job types
- **Time zones**: Store all times in UTC
- **Clock sync**: NTP on all nodes

Trade-offs

Decision	Option A	Option B	Recommendation
Trigger	Polling	Event-driven	**Polling** - simpler
Execution	At-least-once	Exactly-once	**At-least-once + idempotency**
Scheduler	Single leader	Sharded	**Leader** - simpler for <1M jobs
Queue	Redis	Kafka	**Redis** - simpler; Kafka for huge scale

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Work queue for job distribution at scale	Tech Guide

Redis	Work queue and idempotency key storage	Tech Guide
PostgreSQL	Job store and scheduling metadata	Tech Guide

How to Explain It Well

> **Opening**: "Job scheduler needs reliable triggering and at-least-once execution. I'll use optimistic locking for exactly-once triggering."

> **Key talking points**:
>
> - "Optimistic lock: UPDATE WHERE status=pending ensures one scheduler wins"
> - "Work queue decouples scheduling from execution"
> - "Exponential backoff with max retries for transient failures"
> - "Dead letter queue for manual review of persistent failures"
> - "Workers are stateless, can scale horizontally"
